# Analysis of an Avionic System

Vincenzo Esposito - P38000267
Elena d'Agostino - P38000299

## 1 GSPN Exercise – Availability of a Redundant Avionic System

Modern flight control systems must ensure a high level of availability and fault tolerance.

`Availability` is a fundamental property that measures the system's ability to be operational and deliver its intended service over time. Informally, it can be understood as the `probability` that the system is up and running at a given time $t$.

Formally, the availability function $A(t)$ is defined as:

$$A(t) = P(\text{system is operational at time } t) = \frac{T_{\text{up}}}{T_{\text{up}} + T_{\text{down}}},$$

where:

- $T_{\text{up}}$ is the total time the system is functioning correctly,
- $T_{\text{down}}$ is the total time the system is not operational.

This function is bounded between 0 and 1:

$$0 \leq A(t) \leq 1.$$

Availability can also be expressed as a `performance metric`, particularly in systems that are expected to operate continuously (i.e., without regular shutdowns). In such cases, it can be approximated as:

$$A = \frac{\text{MTTF}}{\text{MTTF} + \text{MTTR}} = \frac{\text{MTTF}}{\text{MTBF}},$$

where:

- `MTTF` (Mean Time To Failure) is the expected time the system operates before a failure,

- `MTTR` (Mean Time To Restore) is the average time required to recover from a failure,
- `MTBF` (Mean Time Between Failures) is the sum of MTTF and MTTR.

This quantitative view of availability is important because it is often used in the design and evaluation of safety-critical systems, such as those found in avionics.

In this work, it is analyzed a redundant avionic system composed of three independent processing channels. Each channel receives pilot commands and sends output to an arbitration unit.
The system adopts a "`2-out-of-3`" logic: it is considered safe when at least two channels produce consistent outputs.

However, each channel is subject to potential failures over time. Failures can be:

- `Detectable:` the channel is recognized as faulty and excluded from operation.
- `Undetectable:` the channel continues to provide potentially incorrect output without being detected.

The arbitration unit reacts as follows:

- If two or more functioning channels are available, or if only one correct channel remains while the other two have been safely deactivated, the system operates in a *safe* state.
- If the number or type of faults prevents the arbiter from identifying a reliable output, the system enters a *risk* state.
- If no trustworthy output is available, the system enters a *dangerous* state.

The objective of this assignment is to model the system using a `Stochastic Petri Net (GSPN)` and analyze its dynamic behavior under different failure scenarios.

The following `quantitative analyses` will be considered:

1. Computation of the mean time to reach the dangerous state.
2. Evaluation of the probability that the system remains safe for at least 1000 hours.
3. Sensitivity analysis with respect to parameters $\lambda$ and $c$.
4. Estimation of the probability of silent failure, e.g., the case in which two undetectable faults occur.

## 1.1 Bottom-Up Modeling of the GSPN Architecture

The construction of the GSPN model follows a `bottom-up approach`: the overall system is built by first developing and validating smaller sub-models, which are then integrated into a complete network. This method facilitates modularity and simplifies the modeling process by focusing on well-defined components.

However, it is important to note that the correctness of each individual sub-model does not guarantee the correctness of the overall system. Special attention must be paid during the integration phase to ensure that the interactions between sub-models behave as expected.

In this case, the GSPN is composed of two main sub-models:

- `Device Sub-model:` represents one of the three redundant avionic channels. Each device can transition from a working state to either a *detectable failure* or an *undetectable failure*, based on probabilistic transitions. These three identical sub-models are instantiated to simulate the behavior of all channels in the system.
- `Arbitration Sub-model:` represents the logic of the selector that processes the outputs of the three channels. It evaluates the system state (*safe, at risk, or dangerous*) according to the number and type of working or failed devices, following the 2-out-of-3 logic described previously.

## 1.2 Device Sub-model Description

The first sub-model, representing a single avionic channel, is composed of `four places`, `one timed transition`, and `two immediate transitions`, as illustrated in Figure 1.

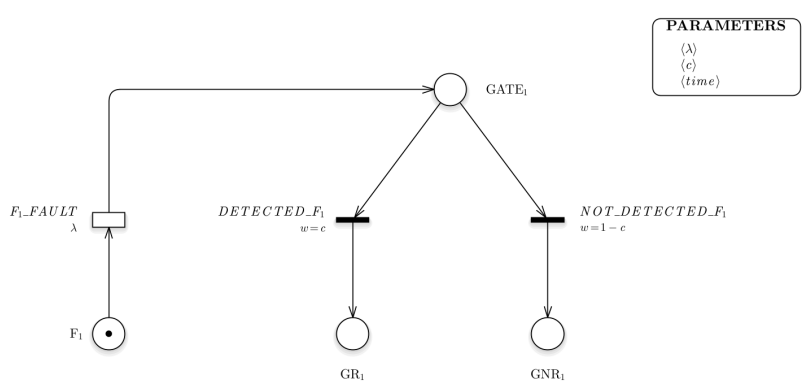


Figure 1: Structure of the Device Sub-model for one avionic channel.

Below is a detailed explanation of the key places in the GSPN sub-model and their roles in the system behaviour:

- `Place` $F_1$ contains one initial token and represents the functioning state of the first channel.
- `Place` $Gate_1$ acts as an intermediate stage, allowing the model to determine whether a failure is detectable or not.
- `Place` $GR_1$ indicates that the channel has experienced a detectable failure.
- `Place` $GNR_1$ indicates that the channel has experienced a non-detectable failure.

The transition `F1_FAULT` is a *timed transition* governed by an exponential distribution with failure rate $\lambda$. When this transition is enabled, it randomly draws a firing time from the negative exponential distribution and fires once the elapsed simulation time reaches that sampled value. This behavior captures the memoryless nature of exponential distributions, modeling the time until failure as a stochastic process.

Since all three devices start in the working state (places $F_1, F_2, F_3$ each contain one token), all three fault transitions (`F1_FAULT`, `F2_FAULT`, `F3_FAULT`) are enabled at the beginning of the simulation. The transition associated with the smallest sampled firing time will fire first, representing the first failure among the devices. The remaining transitions will then be re-evaluated in the next simulation cycle.

Once F1_FAULT fires, it leads to two immediate transitions in conflict: `DETECTED_F1` and `NOT_DETECTED_F1`. This conflict is resolved based on probabilistic weights:

- `DETECTED_F1` is assigned a weight equal to the fault detection capability $c$,
- `NOT_DETECTED_F1` is assigned a weight equal to $1 - c$.

This setup ensures that the outcome of the failure (detectable or not) is chosen randomly according to the defined probability distribution. The sum of the probabilities is equal to 1, ensuring a consistent and realistic fault classification mechanism.

## 1.3 Arbitration Sub-model Description

The second major component of the GSPN is the `arbitration sub-model`, which is further divided into two logical parts:

- A component that determines the specific configuration of the three channels at a given time;
- A component that determines the corresponding state of the system (safe, at risk, or dangerous).

To identify the channel configuration, we use a set of immediate transitions that evaluate the current state of each channel.
Since there are three channels, the total number of possible system configurations is given by:

$$3^3 = 27.$$

The full enumeration of the 27 possible configurations is shown in Table 1. Each row corresponds to a unique combination of channel states and its classification into Safe, Risk, or Dangerous system state.

Table 1: Classification of system states based on the status of the three channels.

| **Channel 1** | **Channel 2** | **Channel 3** | **System State** |
|---|---|---|---|
| F | F | F | Safe |
| F | F | GR | Safe |
| F | GR | F | Safe |
| GR | F | F | Safe |
| GR | GR | F | Safe |
| GR | F | GR | Safe |
| F | GR | GR | Safe |
| F | F | GNR | Safe |
| F | GNR | F | Safe |
| GNR | F | F | Safe |
| F | GR | GNR | Risk |
| F | GNR | GR | Risk |
| GR | F | GNR | Risk |
| GR | GNR | F | Risk |
| GNR | F | GR | Risk |
| GNR | GR | F | Risk |
| F | GNR | GNR | Risk |
| GNR | F | GNR | Risk |
| GNR | GNR | F | Risk |
| GR | GR | GR | Dangerous |
| GR | GR | GNR | Dangerous |
| GR | GNR | GR | Dangerous |
| GR | GNR | GNR | Dangerous |
| GNR | GR | GR | Dangerous |
| GNR | GR | GNR | Dangerous |
| GNR | GNR | GR | Dangerous |
| GNR | GNR | GNR | Dangerous |

What is particularly interesting in this model is the way system states are represented and updated. Temporarily leaving aside the immediate transitions that classify the system configuration (based on the status of each device), we focus here on how the state transitions are managed.

As shown in Figure 2, the safe place initially contains one token. This is coherent with the initial marking of the system, where all three devices are

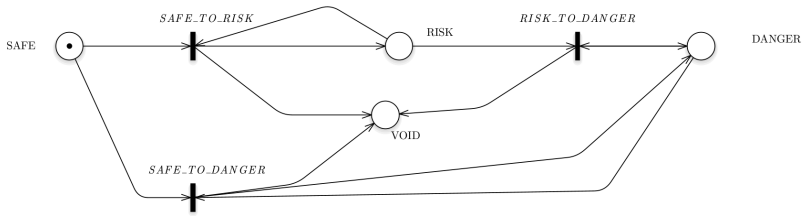


Figure 2: Representation of the system-level state transitions

functioning (i.e., $F_1 = F_2 = F_3 = 1$). Therefore, the system starts in the `SAFE` state.

With a single change in the configuration (e.g., a failure in one channel), the system may still remain in the safe condition. This situation will be correctly handled later when sub-models are merged and synchronized, ensuring consistency of the state update logic.

When an immediate transition determines that the current configuration corresponds to the risk state, the `RISK` place receives a token. A transition labeled `SAFE_TO_RISK` ensures the proper movement of the token from `SAFE` to `RISK`, i.e., it removes the token from `SAFE` and places one in `RISK`.

A similar structure is used for the transition from `RISK` to `DANGER`, through the transition `RISK_TO_DANGER` and the transition from `SAFE` to `DANGER`, through the transition `SAFE_TO_DANGER`.

## 1.4 Integration of the Sub-models

To assemble the complete GSPN, the three identical device sub-models are combined with the arbitration sub-model. The key element that synchronizes these components is a dedicated checker place, hereafter denoted by `CHECKER`. Its purpose is to guarantee that every time a device changes state, the global system state is re-evaluated exactly once.

- Both immediate transitions (`DETECTED Fi` and `NOT DETECTED Fi`) place a token in `CHECKER`. Hence, whenever any channel fails (detected or not), `CHECKER` receives one token, signaling that the overall configuration has changed. The presence of this token allows the `CHECKER` to trigger *only once*, thanks to the fact that the immediate transitions also return tokens to their original places. Without this mechanism, the immediate transitions could fire *indefinitely*, leading to unrealistic behavior. This design ensures a *single reaction per failure event*, maintaining the model's correctness and avoiding infinite firing loops.

- The arbitration sub-model contains 27 *mutually exclusive* immediate transitions, one for each possible combination of channel states ($F$, $GR$, $GNR$). Every configuration transition has:

- *Inputs:*
  ① The `CHECKER` place (exactly one token is required);
  ② The three channel-state places that match the corresponding combination (e.g. transition F_F_GR consumes tokens from $F_1$, $F_2$, and $GR_3$).
- *Outputs:*
  ① Tokens returned to the same channel-state places (to preserve the marking);
  ② A token deposited in the appropriate `SAFE`, `RISK`, or `DANGER` place, depending on the configuration;

Once the 27 configuration transitions are defined, they can be connected to the corresponding global system states: `SAFE`, `RISK`, and `DANGER`. However, a particular case must be handled with care.

In some scenarios, a configuration transition may determine that the system remains in the `SAFE` state, even though the marking has changed (e.g., a detected fault in one channel). This would result in a new token being placed into the `SAFE` place while one is already present, leading to an incorrect accumulation of tokens.

To prevent this behavior, a temporary place called `TEMPORARY_SAFE` is introduced. This place initially receives the token from the configuration transition instead of placing it directly into `SAFE`. A dedicated transition, called `VERIFY_SAFE`, then checks whether both `TEMPORARY_SAFE` and `SAFE` contain a token. If so, it removes one token from each and restores a single token to `SAFE`, while sending the second token to a special place called `VOID`.

The `VOID` place has no further connections and serves only to preserve the `P-invariance` of the net. Without it, the total number of tokens in the state places would not be conserved, and the *P-semiflow* would be violated.

Figure 3 provides an overview of the full GSPN structure, illustrating the interplay between the device sub-models, the arbitration module, and the CHECKER-based state evaluation logic.
The file `Simulation.mp4` contains a video recording of the GSPN simulation, illustrating the step-by-step evolution of the system under fault progression and state transitions.

## 1.5 Structural Analysis of the GSPN Model

The structural analysis of the Petri net allows one to investigate essential properties of the system, regardless of its dynamic behaviour or simulation results. In particular, the concepts of `P-invariants`, `T-invariants`, `boundedness`, and the `reachability graph` play a central role in understanding the fundamental characteristics of the model.

**P-invariants.** It identifies a set of places for which the weighted sum of tokens remains constant across all reachable markings, regardless of the initial

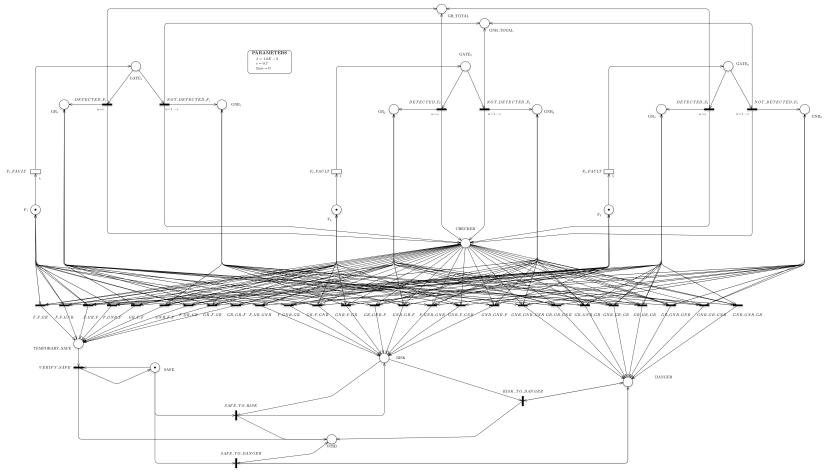


Figure 3: Complete structure of the GSPN model

marking $M_0$. This property is particularly useful to prove that the number of tokens in certain places is bounded—hence, it can be directly used to verify the *boundedness* of the net. Moreover, P-invariants can sometimes be used to infer mutual exclusion conditions among places, though this aspect is not relevant to the current model. It is checkable in P_Semiflows.png file.

Using the `GreatSPN` module, a structural analysis of the model was performed. The results confirm that all places are covered by at least one `P-semi-flow`. This directly implies that every place in the net is bounded, i.e., the number of tokens it can hold is limited across all reachable markings, as can be observed in Boundedness-png file. Thus, the net can be formally classified as *structurally bounded*.

**T-invariants.** A T-invariant is a multiset of transitions whose firing (in any order) leads the net back to the same marking from which it started. This property is useful to detect cyclic behaviours and repetitive operating patterns within the system.

Using `GreatSPN`, a structural analysis of the model reveals that 38 transitions are not covered by any T-semi-flow. This confirms that the net is not T-invariant, as can be shown in T_Semiflows.png file.

However, this result is entirely consistent with the intended behaviour of the system. The model has been explicitly designed *not* to return to its initial marking, since each fault leads the system into a progressively more degraded state. In such non-cyclic systems, the absence of T-invariants is expected and

does not imply an error in the model.

**Reachability Graph.** The reachability graph of a bounded Petri net is a finite representation of all the possible markings reachable from the initial state $M_0$, as shown in Figure 4 and in Reachability_Graph.pdf file. In the case of a bounded GSPN, it is also referred to as the *coverability tree*.

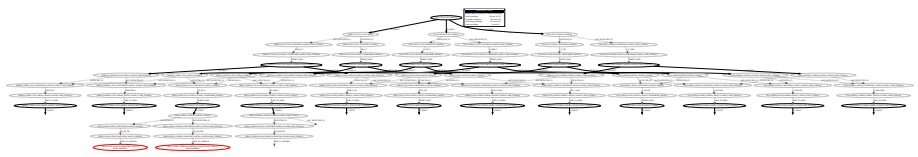

Figure 4: Reachability graph generated from the complete GSPN model.

## 1.6 Mean Time to Dangerous State

The first requirement is to determine the mean time to enter the `Danger` state. Since `GreatSPN` does not provide a built-in tool to directly compute this metric, a simple empirical procedure was adopted using the `SteadyState Simulation` module.

The simulations were carried out using the following parameters:

- $\lambda = 1.0 \times 10^{-4}$: the failure rate associated with each channel, meaning that, on average, a channel fails every 10,000 hours;
- $c = 0.5$: the probability that a fault is correctly detected when it occurs.

The approach followed these steps:

- The simulation was configured to terminate automatically upon reaching a dead marking using the `Tool Statistics` option.
- In this model, all dead markings correspond to the `Danger` state. As a result, the reported simulation time corresponds to the time taken to reach `Danger`.
- To avoid repeated trajectories due to fixed seeds, the random seed was changed manually before each run.
- A total of 20 independent simulation runs were performed, and the resulting times were averaged to estimate the mean time to danger.

Table 2 lists the 20 measured times $t_i$ (in hours) and the corresponding arithmetic mean $\bar{t}$.
The arithmetic mean of the measured times is:

$$\bar{t} = \frac{1}{20} \sum_{i=1}^{20} t_i = 19746.76 \text{ hours}$$

Table 2: Observed times to reach the `Danger` state in 20 simulation runs.

| Run | $t_i$ [h] | Run | $t_i$ [h] |
|---|---|---|---|
| 1 | 27575.2 | 11 | 29850.1 |
| 2 | 17619.6 | 12 | 15208.5 |
| 3 | 32438.6 | 13 | 3762.18 |
| 4 | 14020.9 | 14 | 15870.2 |
| 5 | 17843.1 | 15 | 6602.24 |
| 6 | 25964.5 | 16 | 45364.9 |
| 7 | 10913.7 | 17 | 10463.4 |
| 8 | 59241.7 | 18 | 6718.55 |
| 9 | 10475.3 | 19 | 20568.1 |
| 10 | 15212.6 | 20 | 9221.79 |
| Mean $\bar{t}$ | | 19746.76 | |

This value provides an empirical estimate of the mean time required for the system to reach the `Danger` state under the given parameters.

## 1.7 Probability of Operating in `SAFE` State for at Least 1000 Hours

The next objective is to estimate the probability that the system remains in the `SAFE` state for at least 1000 hours.

This was done with the `Transient Analysis` module, which evaluates performance indices over a finite horizon by repeatedly simulating the Petri net. For each time instant, the tool executes the net a fixed number of iterations (10'000 in our study). During these 10'000 runs it simply counts how many times a given condition is satisfied and, by frequency evaluation, returns the probability of that condition.

In our case, the monitored condition is:

$$M(\texttt{SAFE}) = 1,$$

i.e., exactly one token in place `SAFE`. Starting from the initial marking (which is already `SAFE`), probabilities are sampled at discrete times from 0 to 1000 hours with a step of 100 hours.

This check is meaningful because the system is structured in such a way that:

- From the `SAFE` state, the system can either remain in `SAFE`, or transition to `RISK` (and possibly then to `DANGER`);
- A direct transition from `SAFE` to `DANGER` is also possible.

Therefore, if the system is found in `SAFE` at time $t = 1000\,\text{h}$, it implies that it has never left the `SAFE` state during the entire time horizon. As a consequence, the transient probability:

$$P(M(\texttt{SAFE}) = 1 \mid t = 1000)$$

corresponds directly to the probability that the system operated safely for at least 1000 hours.

The analysis depends on two parameters:

- $\lambda$ — the failure rate of each channel;
- $c$ — the probability that a fault is correctly detected.

As expected, a lower $\lambda$ (rarer failures) increases the chance of remaining in `SAFE` for a fixed period, whereas a higher $\lambda$ reduces it. To illustrate the effect, the detection probability is fixed at $c = 0.7$ while $\lambda$ is varied. Table 3 reports the probabilities of being in `SAFE`, `RISK`, or `DANGER` at $t = 1000$ h for several $\lambda$ values, obtained with 10'000 transient-analysis iterations per point.

Table 3: Probabilities at $t = 1000$ h for different $\lambda$ ($c = 0.7$, 10 000 iterations per point).

| $\lambda$ | $P(\texttt{SAFE})$ | $P(\texttt{RISK})$ | $P(\texttt{DANGER})$ |
|---|---|---|---|
| $1.0 \times 10^{-5}$ | 99.99% | 0.00% | 0.00% |
| $1.0 \times 10^{-4}$ | 99.83% | 0.00% | 0.16% |
| $1.0 \times 10^{-3}$ | 82.59% | 0.00% | 17.40% |
| $1.0 \times 10^{-2}$ | 0.02% | 0.00% | 99.97% |

The table clearly shows the inverse relationship between $\lambda$ and the probability of staying `SAFE`; at higher failure rates the system is increasingly likely to enter `RISK` or `DANGER` within 1000 hours.
An explanation for why the probability of entering the `RISK` state is consistently close to zero will be provided in the following section.

## 1.8 Sensitivity Analysis with Respect to $\lambda$ and $c$

The next task involves performing a sensitivity analysis on the model with respect to the failure rate $\lambda$ and the fault detection probability $c$.

Since `GreatSPN` does not provide a built-in sensitivity analysis tool, a simple but effective method was adopted: by running a series of transient simulations for different values of the parameters, it is possible to observe how the system's behavior changes.

More precisely, the approach adopted consists in performing a set of transient simulations where one parameter is held fixed while the other is varied. The goal is to observe how the system behavior deviates from a reference case.

The reference configuration used for comparison is a transient analysis up to time 10'000 hours, with:

- failure rate $\lambda = 10^{-3}$,

- fault detection probability $c = 0.5$.

Two main sensitivity scenarios are considered:

➢ **Case 1 – $c$ constant, $\lambda$ varies**: in this case, the detection capability remains fixed while the failure rate is changed. As previously reported in Table 3, it was observed that as $\lambda$ increases (i.e., failures become more frequent), the probability of remaining in the `SAFE` state decreases. Conversely, decreasing $\lambda$ leads to a increase in safety, as the system less rapidly transitions toward `RISK` and `DANGER`.

➢ **Case 2 – $\lambda$ constant, $c$ varies in** $[0, 1]$: in this configuration, the failure rate is fixed while the detection capability is progressively improved. This allows a direct assessment of how fault detection quality impacts the overall reliability of the system.

In the following, particular attention is devoted to the second case, where $c$ varies across its entire domain while keeping $\lambda$ fixed.

In this sensitivity study, it was observed that modifying $c$—even across a broad range—had a relatively small or negligible impact on the computed probability of the `RISK` state, which often remained close to zero.

This outcome was further investigated and can be explained by the structure of the system's logic. As previously noted, the system enters a `RISK` state only under specific conditions—particularly, when one channel is functioning and the other two are in different faulty modes: one with a detected fault (`GR`) and the other with an undetected fault (`GNR`). This scenario corresponds to a very limited subset of the 27 possible configurations of the three channels.

To better understand this behavior, it is useful to recall how system states are defined based on the condition of the three channels:

- If two or more channels are `F` (functioning), the system is in the `SAFE` state.
- If one channel is functioning and both of the others are in a `GR` (detected fault) state, the system is still considered `SAFE`.
- If one channel is functioning and both others are in a `GNR` (undetected fault) state, the system enters the `RISK` state.
- If the two faulty channels are in different states—one in `GR`, the other in `GNR`—the system is also considered in `RISK`.
- Finally, if no channel is functioning, the system enters the `DANGER` state.

Given these definitions, the probability of being in the `RISK` state is structurally low when all three devices share the same fault detection probability $c$. In such a case, the system tends to produce symmetric fault outcomes—either both faults are detected (`GR`) or both are not (`GNR`). Therefore, configurations

involving complementary faults (GR + GNR) with one device functioning are statistically rare, and the resulting probability of RISK approaches zero.

However, if the fault detection probabilities $c_1$, $c_2$, and $c_3$ are not all equal—i.e., at least one device has a different detection probability—then the probability of obtaining complementary faults increases. This leads to a noticeable rise in the occurrence of the RISK state throughout the simulation, as the system is more likely to encounter the GR + GNR configuration.

This explains why, under uniform $c$ values, simulations often show the RISK probability as virtually zero, while introducing asymmetry in detection capabilities can make RISK states more prominent.

## 1.9 Probability of Silent Failure (Undetected or Partially Detected Faults)

The final objective is to estimate the probability that the system fails silently. In this context, *silent failure* is defined as reaching the DANGER state without proper fault detection, specifically in the following configurations:

- all three devices are in the GNR state (undetected faults), or
- two devices are in the GNR state and one is in the GR state (detected but unrecovered fault).

To capture this broader failure condition, two dedicated places named GNR_TOTAL and GR_TOTAL were introduced in the model (as illustrated in Figure 3). These places accumulate tokens representing the number of devices currently in the GNR and GR states and are updated by each NOT_DETECTED_F$_\mathtt{i}$ and DETECTED_F$_\mathtt{i}$ transitions.

The silent failure condition is then characterized by the logical formula:

$$(M(\texttt{GNR_TOTAL}) = 3) \quad \text{or} \quad (M(\texttt{GNR_TOTAL}) = 2 \wedge M(\texttt{GR_TOTAL}) = 1)$$

The resulting probability, computed under the symmetric configuration (equal fault detection probability $c$ for all devices), was found to be:

$$P_{\text{silent}} = 0.000107\%$$

During the analysis, it was observed that varying the parameter $c$—which governs the probability that a fault is detected (i.e., the transition toward GR)—influences the steady-state probabilities of the SAFE, RISK, and DANGER markings, without changing the structure of the reachability graph.

Conversely, inverting the weights of the immediate transitions (DETECTED_F$_\mathtt{i}$ and NOT_DETECTED_F$_\mathtt{i}$)—i.e., swapping the values of $c$ and $1 - c$—induces a structural modification in the graph. This redirection of tokens (towards GR or GNR) changes the overall trajectories of the system and affects the final state probabilities, even though the nominal detection rate remains unchanged.

This highlights that the GR and GNR places are not interchangeable: their distinct contributions lead to asymmetric outcomes in system behavior. The

model's topology and dynamic evolution are thus intimately tied to the semantic difference between detection and non-detection, beyond probabilistic equivalence.

# 2 Model Checking Exercise - Logic Control of Fault Tolerances

In this second part of the report, the focus shifts from the structural modeling of the system to its formal verification. Using temporal logic specifications and model checking techniques, we analyze the dynamic behavior of the system to assess whether it meets specific requirements.

We begin by modeling each avionic channel using the following finite state machine (FSM):

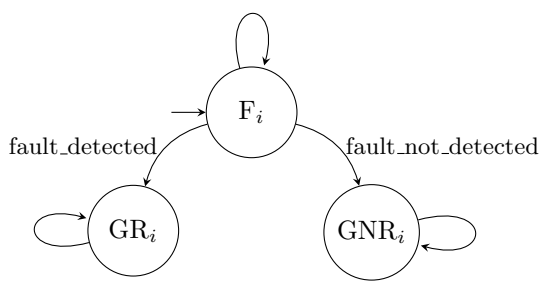


Figure 5: Finite state machine for a single redundant channel.

To determine the overall safety level of the system, we introduce an arbiter that evaluates the combination of channel states and classifies the global condition. Its behavior is captured by the following FSM:

## 2.1 Formal Model of the System in NuSMV

To verify the dynamic behavior of the redundant avionic system and its arbiter, we implement a formal model in NuSMV.

We define a `channel` module to represent the behavior of a single avionic channel. The channel can be in one of three states:

- `functioning` (operating correctly),
- `fault_detected` (fault detected and flagged),
- `fault_not_detected` (fault occurred but not detected).

The state evolves non-deterministically from `functioning` to any of the three states, modeling the possible fault occurrences and detections. Once in a fault state, the channel remains there.

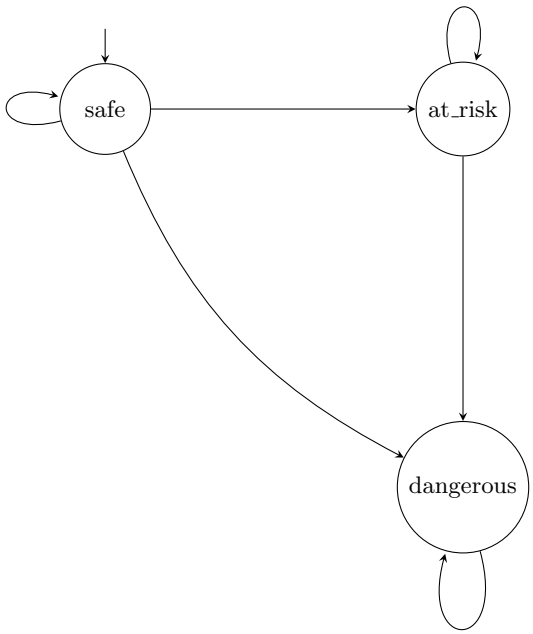


Figure 6: FSM of the arbiter evaluating system safety levels.

```
MODULE channel
VAR
    state : {functioning, fault_detected, fault_not_detected};

ASSIGN
    init(state) := functioning;
    next(state) := case
        state = functioning : {functioning, fault_detected, fault_not_detected};
        TRUE : state;
    esac;
```

The `main` module instantiates three such channels (`c1`, `c2`, `c3`) representing the redundant channels in the system, and an enumerating variable `arb` that models the global safety state classified by the arbiter. We compose the overall system automaton by synchronously composing the three channel automata along with the arbiter automaton, so that all components evolve in lockstep at each step of the system.

The arbiter can be in one of three states: `safe`, `at_risk`, or `dangerous`.

The next state of the arbiter depends on the `next` states of the three channels according to the following rules:

- If all three channels are *not functioning*, the system transitions to the

`dangerous` state.

- If exactly one channel is *functioning*, and the other two channels are either *undetected faults* or a combination of a *detected* and an *undetected* fault, the system is classified as `at_risk`.
- In all other cases — when two or more channels are functioning, or when one channel is functioning and the other two have *detected faults* — the system is considered `safe`.

This classification logic is encoded as follows:

```
MODULE main
VAR
        c1 : channel;
        c2 : channel;
        c3 : channel;
        arb : {safe, at_risk, dangerous};

ASSIGN
        init(arb) := safe;
        next(arb) := case
    (next(c1.state) != functioning & next(c2.state) != functioning
        ↪ & next(c3.state) != functioning) : dangerous;

    (
      ((next(c1.state) = functioning) & (next(c2.state) =
          ↪ fault_detected) & (next(c3.state) = fault_not_detected
          ↪ )) |
      ((next(c1.state) = functioning) & (next(c2.state) =
          ↪ fault_not_detected) & (next(c3.state) = fault_detected
          ↪ )) |
      ((next(c1.state) = fault_detected) & (next(c2.state) =
          ↪ functioning) & (next(c3.state) = fault_not_detected))
          ↪ |
          ((next(c1.state) = fault_not_detected) & (next(c2.state)
              ↪ = functioning) & (next(c3.state) = fault_detected)
              ↪ ) |
          ((next(c1.state) = fault_detected) & (next(c2.state) =
              ↪ fault_not_detected) & (next(c3.state) =
              ↪ functioning)) |
          ((next(c1.state) = fault_not_detected) & (next(c2.state)
              ↪ = fault_detected) & (next(c3.state) = functioning)
              ↪ ) |
          ((next(c1.state) = functioning) & (next(c2.state) =
              ↪ fault_not_detected) & (next(c3.state) =
              ↪ fault_not_detected)) |
          ((next(c1.state) = fault_not_detected) & (next(c2.state)
              ↪ = functioning) & (next(c3.state) =
              ↪ fault_not_detected)) |
          ((next(c1.state) = fault_not_detected) & (next(c2.state)
              ↪ = fault_not_detected) & (next(c3.state) =
              ↪ functioning))
    ) : at_risk;

    TRUE : safe;
```

```
esac;
```

## 2.2 Formal Verification Properties

To formally assess the behavior of the avionic system, we define and verify several temporal logic properties using Computation Tree Logic (CTL).

- **Property 1:**

```
CTLSPEC (
  (c1.state = functioning &
   c2.state = functioning &
   c3.state = functioning)
  -> EG (arb = safe))
```

  This property expresses that if the system starts in a condition where all three channels are functioning, then there exists an execution path where the arbiter remains in the `safe` state indefinitely. In other words, it is possible for the system to maintain safety forever, starting from a fully operational configuration.

- **Property 2:**

```
CTLSPEC AG (
  ((c1.state = fault_not_detected & c2.state =
      ↪ fault_not_detected) |
   (c2.state = fault_not_detected & c3.state =
      ↪ fault_not_detected) |
   (c1.state = fault_not_detected & c3.state =
      ↪ fault_not_detected))
  -> EF (arb = dangerous)
)
```

  This property expresses that in all paths (`AG`), if at least two channels have undetected faults at the same time, then eventually (`EF`) the arbiter will classify the system as `dangerous`.

- **Property 3:**

```
CTLSPEC AG (EF(
  (c1.state = fault_detected |
   c2.state = fault_detected |
   c3.state = fault_detected)
  & (arb = safe)))
```

  This property states that *in every possible state of the system*, there exists a future evolution in which a fault is detected in at least one component and the arbiter eventually enters the `safe` state.

- **Property 4:**

```
CTLSPEC AG !(
  (c1.state != functioning) &
  (c2.state != functioning) &
  (c3.state != functioning) &
  (arb = safe)
)
```

This final property forbids the situation in which all three channels are non-functional and the arbiter still classifies the system as `safe`. That is, it must *never* happen (`AG !`) that complete failure across all channels coexists with a safety status.

## 2.3 Model Checking Results

The following are the results obtained from NuSMV after verifying the CTL properties defined for the system:

```
NuSMV > check_ctlspec
-- specification (((c1.state = functioning & c2.state = functioning
    ↪ ) & c3.state = functioning) -> EG arb = safe)  is true
-- specification AG ((((c1.state = fault_not_detected & c2.state =
    ↪ fault_not_detected) | (c2.state = fault_not_detected & c3.
    ↪ state = fault_not_detected)) | (c1.state =
    ↪ fault_not_detected & c3.state = fault_not_detected)) -> EF
    ↪ arb = dangerous)  is true
-- specification AG (EF (((c1.state = fault_detected | c2.state =
    ↪ fault_detected) | c3.state = fault_detected) & arb = safe))
    ↪  is false
-- as demonstrated by the following execution sequence
Trace Description: CTL Counterexample
Trace Type: Counterexample
  -> State: 1.1 <-
    c1.state = functioning
    c2.state = functioning
    c3.state = functioning
    arb = safe
  -> State: 1.2 <-
    c1.state = fault_detected
    c2.state = fault_detected
    c3.state = fault_not_detected
    arb = dangerous
-- specification AG !(((c1.state != functioning & c2.state !=
    ↪ functioning) & c3.state != functioning) & arb = safe)  is
    ↪ true
```

- The first specification confirms that if all three channels are in the `functioning` state, then it is always possible to maintain the arbiter in a `safe` state indefinitely (`EG arb = safe`). This suggests that a fully functional system can potentially remain in a safe condition.
- The second specification ensures that whenever any two channels do not detect a fault, it is always possible to eventually reach a state where the arbiter signals a dangerous condition (`EF arb = dangerous`).

- The third specification is *false*: it aimed to assert that in every possible state of the system, there exists a future evolution where a fault is detected in at least one channel and the arbiter ultimately reaches a safe state (`AG (EF ((fault_detected) & arb = safe))`). However, the counterexample shows a transition from a fully functioning and safe system (State 1.1), to a state where two channels detect a fault and one does not (State 1.2), but the arbiter becomes `dangerous`. This shows that, from some states, no evolution leads to a `safe` arbiter state after fault detection — thus invalidating the property.

- The fourth specification is `true` and states that the system can never reach a state where all three channels are non-functioning and the arbiter still classifies the system as `safe` (`AG !(((c1.state != functioning & c2.state != functioning) & c3.state != functioning) & arb = safe)`). This ensures that whenever the system is completely degraded, it is never mistakenly classified as safe.